

UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA



Sistemas Operativos

Semestre 2027-1

Tarea 1

Ejercicios de sincronización

Problema del elevador

Profesor: Gunnar Eyal Wolf Iszaevich

Grupo: 7

Integrantes: Carlos Fernando Carballido Carrillo - 320310186

Raúl Haziel Sánchez Soperanes - 321026303

Fecha de entrega: 15 de octubre de 2026

Ciudad Universitaria, Ciudad de México

1 Problema que decidimos resolver

Elegimos el problema del elevador. La idea fue representar con hilos lo que pasa cuando varias personas lo esperan al mismo tiempo y necesitan llegar a pisos distintos. Cada persona tiene su propio hilo y el elevador tiene otro que controla el recorrido.

El planteamiento considera cinco pisos, una capacidad máxima de cinco pasajeros y el paso por todos los pisos intermedios. También indica que una persona puede abordar aunque el elevador vaya en sentido contrario a su destino, siempre que haya espacio [1]. En nuestra implementación numeramos los pisos del 0 al 4 y simulamos a 15 personas con origen y destino aleatorios, siempre distintos.

1.1 Objetivo

Coordinar el ascenso y descenso de los pasajeros con semáforos, respetar la capacidad del elevador y terminar la simulación cuando las 15 personas hayan llegado a su destino.

2 Lenguaje y entorno de desarrollo

Desarrollamos el programa en Java, usando Visual Studio Code en macOS. Todo el código está en `EjercicioElevador.java`. Utilizamos `Random` para generar los viajes y `Semaphore` para coordinar los hilos; ambas clases forman parte de la biblioteca estándar de Java.

2.1 Cómo ejecutar el programa

Se necesita tener instalado un JDK y contar con los comandos `javac` y `java` disponibles en la terminal. No se requieren bibliotecas externas ni instalar Visual Studio Code para ejecutarlo.

Desde la carpeta donde se encuentra el archivo, se compila y ejecuta así:

```
javac EjercicioElevador.java
java EjercicioElevador
```

La salida aparece en la terminal. Muestra las llegadas de las personas, los abordajes, los descensos y un resumen por cada piso visitado. Al finalizar aparece el mensaje `Todos llegaron. El elevador se detiene.` Los viajes cambian entre ejecuciones porque se generan al azar, y el orden de los mensajes también depende de cómo se ejecuten los hilos.

3 Estrategia de sincronización

Usamos exclusión mutua para proteger las cuentas compartidas y señalización para indicar cuándo puede avanzar cada hilo. Además, el elevador espera confirmaciones de los pasajeros antes de continuar el recorrido.

3.1 Datos compartidos y exclusión mutua

Los arreglos `esperando` y `queBajan` guardan, por piso, las personas que esperan y los pasajeros registrados para bajar. La variable `dentro` lleva la ocupación reservada por el elevador y `pendientes` cuenta a quienes todavía no llegan a su destino.

El semáforo `mutex` empieza con un permiso. Cada hilo lo adquiere antes de modificar esos datos y lo libera al terminar. Así evitamos que dos actualizaciones simultáneas pierdan una cuenta. Los contadores de ascenso y descenso se actualizan antes de recibir las confirmaciones, por lo que durante esa etapa representan lugares reservados o liberados.

3.2 Permisos para subir y bajar

Semáforo	Inicial	Uso en el programa
`mutex`	1	Protege las modificaciones del estado compartido.
`semaforoSubir[p]`	0	Una persona espera el permiso para abordar en el piso p.
`semaforoBajar[p]`	0	Un pasajero espera la llegada a su destino p.
`avisoAbordaron`	0	El elevador recibe una confirmación por cada abordaje autorizado.
`avisoBajaron`	0	El elevador recibe una confirmación por cada descenso autorizado.

Una llamada a `acquire()` espera si no hay permisos; `release()` agrega permisos [2]. Por eso una persona queda bloqueada mientras espera el elevador o viaja a su destino, sin revisar continuamente si ya puede avanzar.

3.3 Confirmaciones antes de continuar

En cada parada primero bajan los pasajeros que llegaron a su destino. El elevador libera los permisos correspondientes y espera el mismo número de avisos en `avisoBajaron`. Después calcula cuántos pueden subir:

$$\boxed{\texttt{suben} = \min\big(\texttt{esperando[piso]},\ 5 - \texttt{dentro}\big)}$$

Autoriza esos abordajes y espera sus avisos en `avisoAbordaron`. Esta coordinación es un encuentro o *rendezvous* mediante señalización y confirmación. La espera de los avisos ocurre fuera del `mutex`, lo que permite que los pasajeros adquieran el candado para actualizar sus datos y responder.

4 Funcionamiento del código

4.1 Segmentos 1 a 3 Datos y semáforos

El primer segmento fija cinco pisos, cinco lugares y 15 personas. El segundo reúne los contadores compartidos. En el tercero se declaran el candado, los semáforos de cada piso y los avisos de ascenso y descenso.

Las modificaciones a `pendientes` se hacen dentro del `mutex`. También se declara `volatile` porque el elevador consulta esa variable en la condición del ciclo sin adquirir el candado. Esto da visibilidad a sus actualizaciones; la operación de decremento sigue necesitando la protección del `mutex`.

4.2 Segmento 4 Hilo del elevador

El elevador empieza en el piso 0 y avanza hacia arriba. En cada piso atiende primero los descensos y luego los abordajes. Tras recibir los avisos, imprime el resumen de la parada y espera 300 milisegundos para representar el traslado. Avanza de un piso en un piso e invierte la dirección en los extremos. Sigue recorriendo el edificio mientras haya personas pendientes.

4.3 Segmento 5 Hilo de cada persona

Cada persona guarda su número, origen y destino. Se registra en la espera de su piso y se bloquea hasta obtener permiso para subir. Cuando aborda, registra su destino en `queBajan` y avisa al elevador. Después espera el permiso para bajar. Al llegar, resta una persona de `pendientes` y confirma su descenso.

4.4 Segmento 6 Inicio de la simulación

El método `main` inicializa los semáforos por piso, arranca el hilo del elevador y crea los 15 hilos de personas. Entre cada creación espera un intervalo aleatorio de 0 a 249 milisegundos. Las llegadas se escalonan, pero los hilos que ya se crearon continúan ejecutándose de forma concurrente.

5 Refinamiento y alcance de la solución

Consideramos el refinamiento de evitar la inanición planteado por el profesor [1]. El elevador mantiene un recorrido por todo el edificio y los semáforos de cada piso se crean con `true` para solicitar una política justa.

Esa política ordena la adquisición de permisos según los puntos internos de espera del semáforo [2]. No establece un orden global entre pisos ni garantiza que el número de persona o el mensaje de llegada coincidan con el orden de abordaje. Una persona también puede esperar otra vuelta si el elevador llega lleno.

Con 15 viajes y sin nuevas solicitudes después de ellos, el trabajo pendiente es finito. Suponiendo que los hilos sigan recibiendo tiempo de ejecución y no sean interrumpidos, el recorrido permite completar los viajes. El código no demuestra una garantía general de ausencia de inanición frente a llegadas ilimitadas; para ese caso haría falta una política adicional de asignación de lugares entre pisos.

6 Evidencias de ejecución

Las capturas corresponden a la ejecución en la terminal integrada de Visual Studio Code. Se amplía la zona de mensajes para facilitar su lectura; las imágenes completas se incluyen en el proyecto.

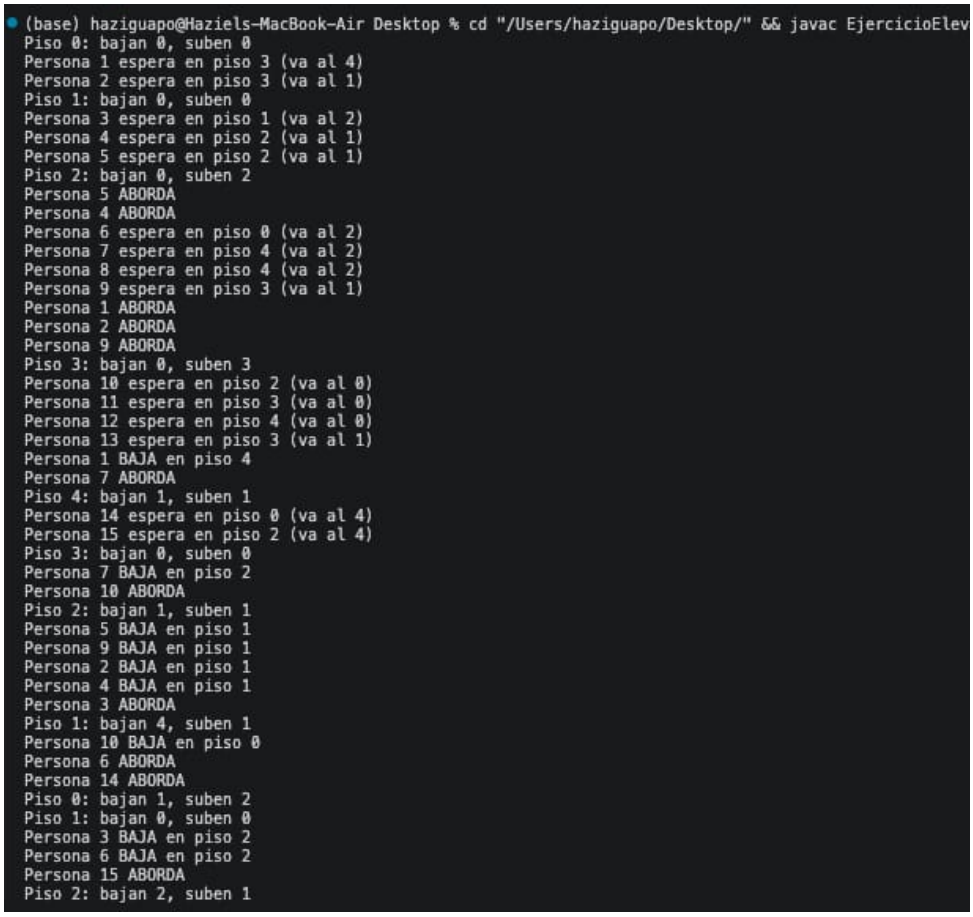


Figura 1: Inicio y desarrollo de la simulación. Se observan llegadas, abordajes y descensos en diferentes pisos.

En la figura 1, la persona 1 espera en el piso 3 y quiere ir al 4. Más adelante aparece su descenso en el piso 4. También se ve que el elevador visita pisos donde no sube ni baja nadie, porque su recorrido pasa por los pisos intermedios.

La salida intercala mensajes de distintos hilos. El aviso de abordaje se envía antes de imprimir `ABORDA`; por eso el resumen de una parada puede aparecer antes que alguno de esos mensajes. La coordinación depende de los semáforos, no del orden visual de las líneas.

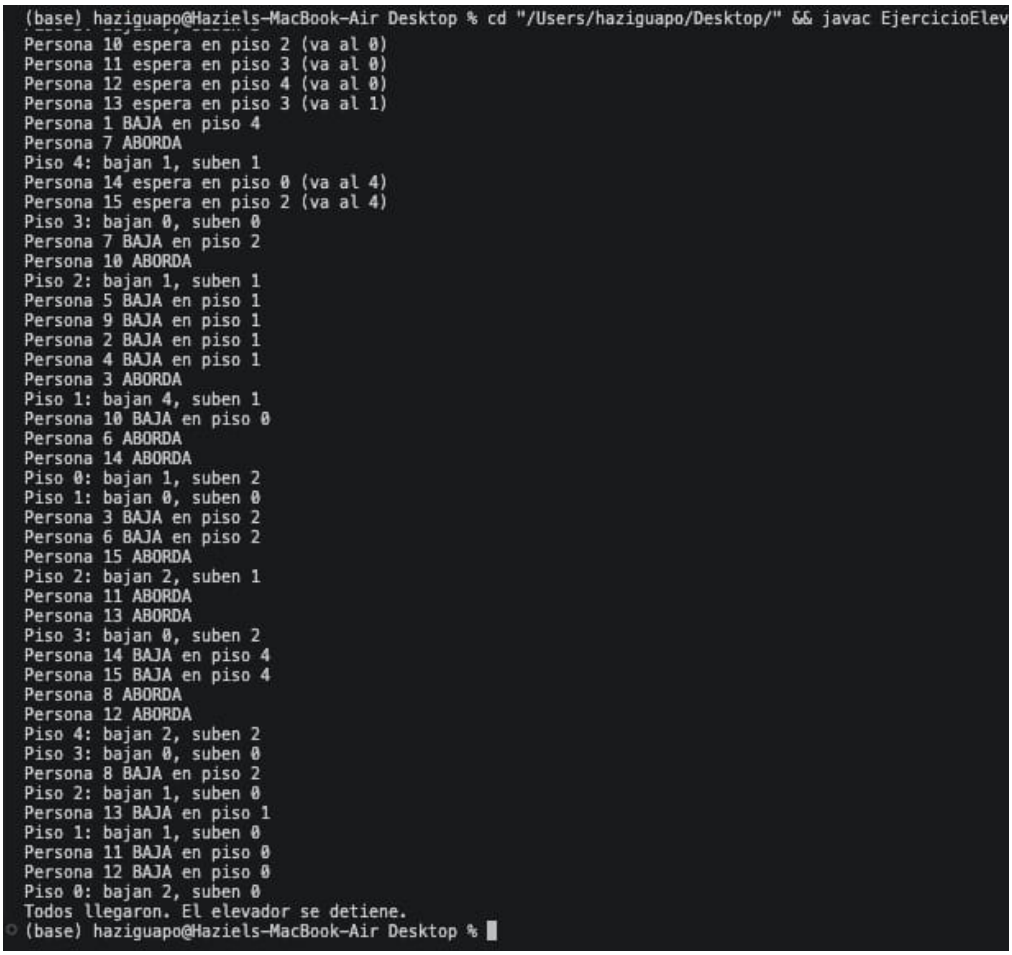


Figura 2: Parte final de la ejecución. Después de los últimos descensos aparece el mensaje de término y vuelve el indicador de la terminal.

La figura 2 muestra que esta ejecución llegó al mensaje final. En la última parada visible bajan dos personas en el piso 0. La condición de salida del elevador depende de que `pendientes` llegue a cero, y ese contador disminuye cuando cada pasajero registra su descenso.

Las capturas documentan una ejecución terminada. La capacidad máxima se controla en el cálculo de los abordajes autorizados; la salida actual no imprime la ocupación total después de cada parada. Agregar ese dato facilitaría revisar visualmente el límite de cinco pasajeros.

7 Dudas y posibles mejoras

El programa conserva algunos puntos que podríamos trabajar después:

- Cuando no hay personas esperando ni viajando, el elevador sigue moviéndose si todavía quedan personas por aparecer. Podría esperar una señal de nueva solicitud para evitar esos recorridos vacíos.
- Los bloques `catch` de los hilos están vacíos. Si un hilo fuera interrumpido, podría terminar sin enviar un aviso que otro hilo espera. Haría falta definir cómo cancelar la simulación y liberar los recursos de forma ordenada.
- El modo justo se aplica a los semáforos por piso; el `mutex` se crea sin esa opción. Queda abierta la cuestión de cómo asegurar una atención justa entre pisos si se recibieran solicitudes de manera continua.
- Una semilla configurable para `Random` permitiría repetir los mismos viajes, aunque el orden de ejecución de los hilos seguiría pudiendo variar. También sería útil imprimir la ocupación y conservar una bitácora de cada recorrido.

Una decisión que mantuvimos fue permitir que alguien suba aunque vaya en sentido contrario a su destino. Esto puede alargar su viaje, pero corresponde a la regla del ejercicio. También nos queda como duda si conviene sustituir la consulta de `pendientes` por una señal explícita de terminación al ampliar el programa.

8 Conclusión conjunta

La parte que tuvimos que coordinar fue el momento en que cada persona podía subir o bajar y cuándo podía continuar el elevador. El `mutex` protege las cuentas, mientras que los permisos por piso y las confirmaciones mantienen el orden de esas acciones. Si el elevador avanzara sin esperar el registro del destino de quienes abordaron, podría llegar al siguiente piso antes de tener completa esa información.

La ejecución presentada termina después de atender a las personas de la simulación. La solución permite revisar cómo se combinan la exclusión mutua y la señalización en un problema concreto. Para ampliarla a solicitudes continuas, revisaríamos primero la atención entre pisos y el manejo de interrupciones.

Referencias

[1] Wolf, G. (s. f.). *Administración de procesos: Ejercicios de sincronización* [Diapositivas, apartado El elevador]. `https://gwolf.sistop.org/laminas/06b-ejercicios-sincronizacion.pdf`

[2] Oracle. (s. f.). *Semaphore*. Java Platform Standard Edition API Specification. `https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Semaphore.html`

A Código fuente completo

```java
import java.util.Random;
import java.util.concurrent.Semaphore;

public class EjercicioElevador {

    // SEGMENTO 1: Datos del problema
    static final int PISOS = 5;
    static final int CAPACIDAD = 5;
    static final int TOTAL_PERSONAS = 15;

    // SEGMENTO 2: Estado compartido (lo protege el mutex)
    static int[] esperando = new int[PISOS];   // cuantos esperan en cada piso
    static int[] queBajan = new int[PISOS];    // cuantos bajan en cada piso
    static int dentro = 0;                     // pasajeros dentro del elevador
    static volatile int pendientes = TOTAL_PERSONAS;  // personas que aun no llegan

    // SEGMENTO 3: Semaforos
    static Semaphore mutex = new Semaphore(1);                  // un hilo a la vez en los datos
    static Semaphore[] semaforoSubir = new Semaphore[PISOS];    // esperan permiso para subir
    static Semaphore[] semaforoBajar = new Semaphore[PISOS];    // esperan llegar a su piso
    static Semaphore avisoAbordaron = new Semaphore(0);         // "ya subi"
    static Semaphore avisoBajaron = new Semaphore(0);           // "ya baje"

    // SEGMENTO 4: Hilo del elevador
    static class Elevador implements Runnable {
        public void run() {
            try {
                int piso = 0, direccion = 1;          // direccion: 1 sube, -1 baja
                while (pendientes > 0) {

                    // Bajar a quienes llegaron a este piso
                    mutex.acquire();
                    int bajan = queBajan[piso];
                    queBajan[piso] = 0;
                    dentro = dentro - bajan;
                    mutex.release();
                    semaforoBajar[piso].release(bajan);   // despierta a los que bajan
                    avisoBajaron.acquire(bajan);          // espera a que bajen todos

                    // Subir a quienes esperan, sin pasar la capacidad
                    mutex.acquire();
                    int suben = Math.min(esperando[piso], CAPACIDAD - dentro);
                    esperando[piso] = esperando[piso] - suben;
                    dentro = dentro + suben;
                    mutex.release();
                    semaforoSubir[piso].release(suben);   // despierta a los que suben
                    avisoAbordaron.acquire(suben);        // espera a que suban todos

                    System.out.println("Piso " + piso + ": bajan " + bajan + ", suben " + suben);

                    // Moverse al siguiente piso (cruza todos los intermedios)
                    Thread.sleep(300);
                    if (piso + direccion < 0 || piso + direccion >= PISOS) {
                        direccion = -direccion;
                    }
                    piso = piso + direccion;
                }
                System.out.println("Todos llegaron. El elevador se detiene.");
            } catch (InterruptedException e) { }
        }
```

```
    }

    // SEGMENTO 5: Hilo de cada persona
    static class Persona implements Runnable {
        int numero, origen, destino;

        Persona(int numero, int origen, int destino) {
            this.numero = numero;
            this.origen = origen;
            this.destino = destino;
        }

        public void run() {
            try {
                // Llegar al piso y esperar el elevador
                System.out.println("Persona " + numero + " espera en piso " + origen + " (va al " + destino +
  ")");
                mutex.acquire();
                esperando[origen]++;
                mutex.release();
                semaforoSubir[origen].acquire();          // bloqueada hasta que la dejen subir

                // Abordar (sin importar la direccion del elevador)
                mutex.acquire();
                queBajan[destino]++;
                mutex.release();
                avisoAbordaron.release();
                System.out.println("Persona " + numero + " ABORDA");

                // Viajar hasta el destino y bajar
                semaforoBajar[destino].acquire();
                System.out.println("Persona " + numero + " BAJA en piso " + destino);
                mutex.acquire();
                pendientes--;
                mutex.release();
                avisoBajaron.release();
            } catch (InterruptedException e) { }
        }
    }

    // SEGMENTO 6: Programa principal
    public static void main(String[] args) throws InterruptedException {
        for (int i = 0; i < PISOS; i++) {
            semaforoSubir[i] = new Semaphore(0, true);   // true = FIFO (evita inanicion)
            semaforoBajar[i] = new Semaphore(0, true);
        }

        new Thread(new Elevador()).start();

        Random azar = new Random();
        for (int i = 1; i <= TOTAL_PERSONAS; i++) {
            int origen = azar.nextInt(PISOS);
            int destino = azar.nextInt(PISOS);
            while (destino == origen) {
                destino = azar.nextInt(PISOS);
            }
            new Thread(new Persona(i, origen, destino)).start();
            Thread.sleep(azar.nextInt(250));             // llegan una tras otra
        }
    }
}
```